STI V2.0 BY A'e

LE MANUEL CONNECTÉ

LYCÉE RAFÈHA

3ème SI

STI

Systèmes et technologies de l’informatique

HTML5 / CSS3 / JAVASCRIPT

BASES DE DONNÉES / SQL

Cours • Méthodes • Travaux pratiques

50 fiches reliées à leur contenu en ligne

Enseignant

Aymen Essouyah

SMART BOOK

Lire. Scanner. Pratiquer.

SCANNER POUR APPRENDRE

ÉDITION CONNECTÉE · 2026

COURS · EXERCICES · RÉVISIONS

Sommaire

MODE D’EMPLOI DU LIVRE CONNECTÉ

Sur écran : cliquez sur le titre du sommaire ou sur « Ouvrir sur STI V2.0 BY A'e ». Sur papier : scannez le QR contrasté au bas de la fiche pour ouvrir la partie correspondante. Une connexion Internet est nécessaire.

Depuis une photo enregistrée : utilisez la détection de QR de votre téléphone. La reconnaissance automatique du texte d’une page, sans QR lisible, n’est pas intégrée. Le filigrane pâle est une signature discrète.

HTML5

01. Structure d’un document HTML5 ··· 5

02. Syntaxe, attributs et imbrication ··· 6

03. Texte, liens et contenus multimédias ··· 7

04. Images cliquables : map et area ··· 8

05. Listes, tableaux et structure sémantique ··· 9

06. Formulaires : saisie et transmission ··· 10

07. Formulaires : options et aide-mémoire ··· 11

08. Cadres intégrés et cibles de navigation ··· 12

09. Attributs globaux, datalist et événements HTML ··· 13

CSS3

10. Syntaxe et intégration des styles ··· 14

11. Sélecteurs, classes et identifiants ··· 15

12. Unités et modèle de boîte ··· 16

13. Texte, couleurs, bordures et arrière-plans ··· 17

14. Positionnement : relative et absolute ··· 18

15. Positionnement : fixed et sticky ··· 19

16. Mise en page avec float et clear ··· 20

17. Flexbox, affichage et débordement ··· 21

18. Liens, listes, tableaux et ombres ··· 22

19. Filtres et transformations 2D ··· 23

20. Transitions et animations ··· 24

Sommaire — suite

CSS3

21. Transitions : durée, délai et progression ··· 25

22. Animations : répétition, direction et remplissage ··· 26

JavaScript

23. Intégration, variables et types ··· 27

24. Opérateurs, conversions et dialogues ··· 28

25. Conditions et choix multiples ··· 29

26. Boucles et accumulation ··· 30

27. TP : jeu de devinettes ··· 31

28. DOM : sélectionner et modifier des éléments ··· 32

29. DOM : lire et contrôler un formulaire ··· 33

30. Fonctions, portée et événements ··· 34

31. Nombres, chaînes et méthodes usuelles ··· 35

32. Tableaux et parcours ··· 36

33. Dates, temporisation et horloge numérique ··· 37

34. Compléments : Math, chaînes et sorties ··· 38

35. Médias, styles et dates de formulaire ··· 39

Bases de données et SQL

36. Notions relationnelles et contraintes ··· 40

37. SQL : commandes et types MySQL ··· 41

38. Création des tables : gestion des locations ··· 42

39. Modifier et supprimer une structure ··· 43

40. Insérer, modifier et supprimer des lignes ··· 44

Sommaire — suite

Bases de données et SQL

41. Base école : tables et données des exercices ··· 45

42. SELECT simple : choisir les colonnes ··· 46

43. WHERE : comparaisons, logique et valeurs absentes ··· 47

44. BETWEEN, IN, LIKE et ORDER BY ··· 48

Travaux pratiques

45. TP 1 : mini-site du Lycée Rafèha ··· 49

46. TP 2 : formulaire contrôlé avec JavaScript ··· 50

47. TP 3 : sélection simple — énoncé et corrigé ··· 51

Annexes

48. Aide-mémoire HTML5, CSS3 et JavaScript ··· 52

49. SQL : bilan de niveau et auto-évaluation ··· 53

50. Sources et utilisation du livre connecté ··· 54

Structure d’un document HTML5

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Objectif

Créer un document HTML valide et comprendre le rôle de ses éléments principaux. HTML structure le contenu ; CSS gère sa présentation ; JavaScript ajoute des interactions.

Document de départ

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <meta name="description" content="Cours de STI">
  <title>Ma première page</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
  <h1>Bienvenue au Lycée Rafèha</h1>
  <p>Une page structurée en HTML5.</p>
</body>
</html>
```

Rôle des éléments

• <!DOCTYPE html> active le mode standard du navigateur.

• html est l’élément racine. lang="fr" indique la langue du contenu.

• head contient le titre et les métadonnées. title est affiché dans l’onglet du navigateur.

• meta charset="utf-8" définit l’encodage ; link relie la feuille de style.

• body contient le contenu de la page. Les couleurs et les images de fond se règlent en CSS, pas avec l’ancien attribut background.

• defer reporte l’exécution du script externe après l’analyse du HTML.

À réaliser

Enregistrer ce document sous index.html, créer style.css et script.js dans le même dossier, puis ouvrir la page dans le navigateur.

Syntaxe, attributs et imbrication

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Éléments et attributs

Une balise ouvrante peut porter des attributs. Les éléments non vides ont une balise fermante. Respecter l’imbrication : le dernier élément ouvert est le premier fermé.

```
<p class="important" id="intro" title="Présentation">
  Un texte <strong>important</strong>.
</p>
<a href="contact.html">Nous contacter</a>
<img src="lycee.jpg" alt="Entrée du lycée" width="400">
```

• id identifie un seul élément dans le document.

• class contient un ou plusieurs noms de classes séparés par des espaces.

• title apporte une information complémentaire ; il ne remplace pas un libellé accessible.

• style permet un style local ; privilégier une feuille CSS externe.

Éléments vides

br, hr, img, input, meta, link et source n’ont pas de balise fermante en HTML. Le slash final de
 est facultatif et ne transforme pas l’élément en paire de balises.

Casse et caractères réservés

Les noms des balises et des attributs HTML ne distinguent généralement pas les majuscules des minuscules. En revanche, les valeurs de id et de class doivent conserver leur casse pour les sélecteurs CSS et les accès JavaScript.

```
<p>Écrire &lt;p&gt; pour afficher le nom de la balise.</p>
<p>Utiliser &amp; pour afficher une esperluette.</p>
<!-- Ceci est un commentaire HTML -->
```

Bonnes pratiques

• Employer des noms simples : menu-principal, titre, carte.

• Indenter le code et entourer les valeurs d’attributs de guillemets droits.

• Ne pas imbriquer un paragraphe dans un autre paragraphe.

• Préférer les éléments sémantiques aux éléments génériques lorsque leur rôle correspond au contenu.

Application

Créer deux paragraphes de même classe et leur attribuer deux identifiants différents. Expliquer pourquoi deux éléments ne doivent pas partager le même id.

Texte, liens et contenus multimédias

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Structurer le texte

h1 à h6 représentent des titres hiérarchisés ; p un paragraphe ; br un saut de ligne ; hr une séparation thématique. Choisir un titre selon son niveau, et non selon sa taille visuelle.

• strong : importance forte ; em : emphase ; mark : passage mis en évidence.

• abbr : abréviation ; sup et sub : exposant et indice.

• blockquote : citation en bloc ; q : citation courte ; cite : titre d’une œuvre.

• del et ins : contenu supprimé ou ajouté ; pre et code : code préformaté.

• address : coordonnées de contact ; time : date ou heure.

```
<h1>Le Web</h1>
<h2>Les langages</h2>
<p><abbr title="HyperText Markup Language">HTML</abbr>
structure la page. CSS gère sa <em>présentation</em>.</p>
<p>H<sub>2</sub>O ; x<sup>2</sup></p>
<time datetime="2026-10-21">21 octobre 2026</time>
<a href="https://developer.mozilla.org/fr/">Documentation Web</a>
<a href="#suite">Aller à la suite</a>
<h2 id="suite">Suite du cours</h2>
```

Images, audio et vidéo

```
<figure>
  <img src="schema.png" alt="Échanges client-serveur">
  <figcaption>Le navigateur dialogue avec le serveur.</figcaption>
</figure>
<audio controls>
  <source src="cours.mp3" type="audio/mpeg">
  Votre navigateur ne peut pas lire ce fichier audio.
</audio>
<video controls width="480">
  <source src="demo.mp4" type="video/mp4">
  Votre navigateur ne peut pas lire cette vidéo.
</video>
```

À retenir

alt décrit une image informative ; alt="" convient à une image purement décorative. Pour une vidéo pédagogique, prévoir des sous-titres. Un lien doit avoir un intitulé compréhensible hors contexte.

Images cliquables : map et area

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Définition

Une image mappée contient plusieurs zones cliquables, chacune reliée à une destination. L’attribut usemap de img désigne le name de map, précédé de #.

```
<img src="plan.png" alt="Plan du lycée" width="400" height="200"
     usemap="#plan-lycee">
<map name="plan-lycee">
  <area shape="rect" coords="10,10,130,90"
        href="bibliotheque.html" alt="Bibliothèque">
  <area shape="circle" coords="250,100,40"
        href="laboratoire.html" alt="Laboratoire informatique">
  <area shape="poly" coords="300,10,390,10,390,80,350,110"
        href="administration.html" alt="Administration">
</map>
```

Formes et coordonnées

rect : x1,y1,x2,y2 pour deux coins opposés. circle : x,y,rayon. poly : suite de couples x,y définissant un polygone. Les coordonnées sont exprimées en pixels CSS dans le repère de l’image ; vérifier leur comportement si l’image est redimensionnée.

Accessibilité

Chaque area possède un texte alternatif qui indique la destination. Proposer également des liens textuels ordinaires sous l’image. Vérifier la navigation au clavier et les zones tactiles : une région trop petite est difficile à sélectionner sur téléphone.

```
<p>Accès direct :
  <a href="bibliotheque.html">Bibliothèque</a> –
  <a href="laboratoire.html">Laboratoire informatique</a> –
  <a href="administration.html">Administration</a>
</p>
```

Application

Dessiner un plan de 400 × 200 pixels avec trois zones, relever les coordonnées puis créer les liens. Tester les clics au centre et près des limites. Comparer les attributs id, name et usemap : ils ne sont pas interchangeables.

Listes, tableaux et structure sémantique

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Les listes

```
<ul><li>HTML5</li><li>CSS3</li></ul>
<ol><li>Créer la page</li><li>Tester</li></ol>
<dl>
  <dt>DOM</dt><dd>Représentation de la page en objets.</dd>
</dl>
```

ul crée une liste non ordonnée ; ol une liste ordonnée ; li un élément de liste. dl regroupe des termes dt et leurs descriptions dd.

Les tableaux de données

```
<table>
  <caption>Résultats des élèves</caption>
  <thead>
    <tr><th scope="col">Nom</th><th scope="col">Note</th></tr>
  </thead>
  <tbody>
    <tr><td>Amal</td><td>16</td></tr>
    <tr><td>Yassine</td><td>14</td></tr>
  </tbody>
</table>
```

tr définit une ligne ; th une cellule d’en-tête ; td une cellule de données. thead, tbody et tfoot regroupent les lignes. colspan et rowspan permettent des fusions de cellules. Ne pas utiliser un tableau pour mettre en page tout le site.

Organiser une page

```
<header><h1>Mon site</h1></header>
<nav><a href="index.html">Accueil</a></nav>
<main>
  <section>
    <h2>Actualités</h2>
    <article><h3>Projet STI</h3><p>Présentation...</p></article>
  </section>
  <aside>Informations complémentaires</aside>
</main>
<footer>Contact du lycée</footer>
```

header : en-tête ; nav : navigation ; main : contenu principal ; section : section thématique ; article : contenu autonome ; aside : complément ; footer : pied de page.

Éléments génériques

Par défaut, div est affiché en bloc et span en ligne. Ils servent à regrouper du contenu sans lui attribuer de signification sémantique particulière. Ils restent utiles même sans attribut CSS.

Formulaires : saisie et transmission

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Formulaire complet

```
<form id="inscription" method="post" action="inscrire.html">
  <fieldset>
    <legend>Identification</legend>
    <label for="nom">Nom</label>
    <input type="text" id="nom" name="nom"
           maxlength="50" required>
    <label for="email">Adresse électronique</label>
    <input type="email" id="email" name="email" required>
    <label for="naissance">Date de naissance</label>
    <input type="date" id="naissance" name="naissance">
    <label for="classe">Classe</label>
    <select id="classe" name="classe">
      <option value="4SI1">4ème SI 1</option>
      <option value="4SI2">4ème SI 2</option>
    </select>
    <label for="message">Message</label>
    <textarea id="message" name="message" rows="4"></textarea>
    <button type="submit">Envoyer</button>
    <button type="reset">Réinitialiser</button>
  </fieldset>
</form>
```

Comprendre les attributs

• action désigne l'URL de traitement ; method choisit get ou post. Leur présence explicite améliore la lisibilité, sans être toujours obligatoire en HTML.

• name est la clé utilisée pour transmettre la valeur au serveur ; id sert notamment à l'association avec label for.

• required rend la saisie obligatoire côté navigateur ; maxlength limite la longueur ; placeholder donne une indication, sans remplacer label.

• min, max et step règlent les bornes et le pas d'un champ number.

• readonly interdit la modification mais conserve l'envoi ; disabled désactive le champ et l'exclut de l'envoi.

GET ou POST ?

GET place les paramètres dans l'URL et convient notamment aux recherches. POST transmet les données dans le corps de la requête et convient aux modifications. POST ne chiffre pas les données : utiliser HTTPS.

Important

Les validations HTML et JavaScript facilitent la saisie mais ne remplacent jamais la validation sur le serveur.

Formulaires : options et aide-mémoire

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Cases à cocher et boutons radio

```
<fieldset>
  <legend>Langages étudiés</legend>
  <label><input type="checkbox" name="langages[]"
                value="html" checked> HTML</label>
  <label><input type="checkbox" name="langages[]"
                value="javascript"> JavaScript</label>
</fieldset>
<fieldset>
  <legend>Groupe</legend>
  <label><input type="radio" name="groupe"
                value="A" required> A</label>
  <label><input type="radio" name="groupe" value="B"> B</label>
</fieldset>
```

Les radios de même name sont exclusifs. Les cases à cocher autorisent plusieurs choix ; elles peuvent partager un nom de tableau, par exemple langages[], pour leur traitement ultérieur sur un serveur. Une case non cochée n’est normalement pas transmise.

Liste multiple et groupes

```
<select name="options[]" multiple size="4">
  <optgroup label="Web">
    <option value="html" selected>HTML</option>
    <option value="css">CSS</option>
  </optgroup>
</select>
<input type="tel" name="telephone" pattern="[0-9]{8}">
<input type="number" name="note" min="0" max="20" step="0.5">
```

Autres types de champs

• password : saisie masquée, mais non chiffrée par ce seul attribut.

• search : recherche ; url : adresse Web ; tel : téléphone.

• range : curseur ; color : couleur ; date : date ; file : fichier.

• hidden : valeur invisible dans l’interface, mais modifiable par l’utilisateur.

Envoi de fichier

Utiliser method="post" et enctype="multipart/form-data" sur form, ainsi qu’un input type="file". Le stockage et la vérification réelle du fichier demandent un traitement serveur, non étudié ici ; ce formulaire montre seulement le contrôle HTML.

Application

Construire un formulaire avec une adresse électronique, une note de 0 à 20, un groupe et plusieurs langages. Vérifier les données transmises dans les outils réseau du navigateur.

Cadres intégrés et cibles de navigation

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

iframe : intégrer un document

iframe affiche une page dans une autre. title fournit un nom accessible au cadre ; name permet de le désigner comme cible. id et name n’ont pas le même rôle.

```
<nav>
  <a href="inscription.html" target="cadre">Inscription</a>
  <a href="liste.html" target="cadre">Liste des élèves</a>
</nav>
<iframe name="cadre" id="cadre" title="Espace de travail"
        src="inscription.html"></iframe>

iframe { width: 100%; min-height: 500px; border: 0; }
```

target sur les liens et formulaires

• _self : contexte courant ; valeur usuelle par défaut.

• _blank : nouveau contexte, généralement un nouvel onglet.

• _parent : contexte parent ; _top : contexte de navigation supérieur.

• cadre : contexte portant ce nom, ici l’iframe de la page.

```
<form action="ajouter.html" method="post" target="cadre">
  <label for="nom">Nom</label>
  <input id="nom" name="nom" required>
  <button type="submit">Ajouter</button>
</form>
<a href="https://developer.mozilla.org/fr/" target="_blank"
   rel="noopener">Documentation Web</a>
```

Si le formulaire est déjà affiché dans cadre, _self permet aussi d’afficher sa réponse dans ce cadre. Si aucun contexte nommé ne correspond à une cible, le navigateur peut en ouvrir un nouveau.

Sécurité et limites

Une page tierce peut interdire son intégration par des en-têtes de sécurité. La politique de même origine limite les accès JavaScript entre documents de domaines différents. sandbox permet de restreindre un cadre ; ses permissions doivent être choisies selon le besoin. Ne pas ajouter toutes les permissions sans les comprendre.

Application

Créer index.html avec une navigation et un cadre nommé cadre. Afficher successivement deux pages locales. Pour une démonstration purement locale, intercepter le formulaire avec preventDefault et afficher une confirmation ; aucun enregistrement serveur n’est demandé.

Attributs globaux, datalist et événements HTML

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Attributs globaux utiles

id identifie ; class regroupe ; title apporte une information ; lang indique une langue ; hidden masque un contenu ; data-* stocke des données propres à l'application. tabindex influence la navigation au clavier : éviter les valeurs positives pour ne pas perturber l'ordre naturel.

```
<article class="produit" data-code="P01" lang="fr">
  <h2>Un produit</h2>
</article>
<label for="ville">Ville</label>
<input id="ville" name="ville" list="villes">
<datalist id="villes">
  <option value="Tunis"></option>
  <option value="Sfax"></option>
  <option value="Sousse"></option>
</datalist>
```

datalist propose des suggestions sans interdire une autre saisie. select propose une liste de choix. Pour imposer une ville reconnue, contrôler sa valeur côté serveur.

Événements dans les attributs

```
<button type="button" onclick="saluer()">Bonjour</button>
<form action="traitement.html" method="post"
      onsubmit="return valider()">
  <input name="nom" required>
  <button type="submit">Envoyer</button>
</form>
```

Dans cet exemple, le return de onsubmit transmet le résultat de valider : false annule l'action par défaut. Écrire seulement onsubmit="valider()" ne transmet pas automatiquement ce false. Avec addEventListener, employer event.preventDefault() pour bloquer l'envoi.

Correspondance des noms

onclick correspond à click ; onchange à change ; oninput à input ; onsubmit à submit. input réagit au changement pendant la saisie ; change intervient lors de la validation d'un changement selon le contrôle. Éviter de multiplier les gestionnaires dans le HTML : addEventListener sépare mieux structure et comportement.

Application

Construire une zone de saisie avec suggestions et un message de validation. Conserver un libellé visible ; ne pas faire dépendre les consignes uniquement de title ou de placeholder.

Syntaxe et intégration des styles

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Définition

CSS (Cascading Style Sheets) décrit la présentation des éléments HTML. Une règle associe un sélecteur à des déclarations propriété: valeur.

```
p {
  color: #16324f;
  font-size: 1rem;
  line-height: 1.6;
}
```

Trois modes d’intégration

```
<!-- Feuille externe : méthode à privilégier -->
<link rel="stylesheet" href="style.css">
<!-- Style interne, dans head -->
<style>
  h1 { color: navy; }
</style>
<!-- Style local -->
<p style="color: navy;">Un paragraphe</p>
```

Cascade, héritage et priorité

Plusieurs règles peuvent viser un même élément. Pour les déclarations ordinaires d’une même origine et d’une même couche, la spécificité du sélecteur puis l’ordre d’écriture départagent les règles concurrentes. Un identifiant est plus spécifique qu’une classe, qui est plus spécifique qu’un nom de balise.

Certaines propriétés sont héritées, comme color et font-family. Les marges et les bordures ne le sont généralement pas. Éviter l’emploi systématique de !important.

```
body { font-family: Arial, sans-serif; color: #222; }
p { color: blue; }
.important { color: darkred; }
#intro { color: darkgreen; }
/* Un p ayant class="important" et id="intro" sera vert. */
```

Règles de syntaxe

• Terminer chaque déclaration par un point-virgule.

• Ne pas séparer la valeur et son unité : 150px, et non 150 px.

• Écrire les commentaires entre /* et */.

• Une règle CSS ordinaire commence par un sélecteur ; les règles commençant par @, comme @media, suivent une syntaxe particulière.

Application

Relier une feuille externe à la page d’accueil et définir les styles du corps, des titres et des paragraphes.

Sélecteurs, classes et identifiants

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Principaux sélecteurs

• p : tous les paragraphes ; * : tous les éléments.

• .important : les éléments portant la classe important.

• #intro : l'élément portant l'identifiant intro.

• h1, h2 : réunion de plusieurs sélecteurs.

• nav a : liens descendants de nav ; nav > a : liens enfants directs.

• input[type="email"] : champs ayant cet attribut et cette valeur.

• a:hover : lien survolé ; input:focus : champ ayant le focus.

```
<h1 id="titre" class="grand souligne">Le Web</h1>
<p class="important">À retenir</p>
<p class="grand important">Un autre paragraphe</p>

#titre { color: #16324f; }
.grand { font-size: 1.5rem; }
.souligne { text-decoration: underline; }
.important { font-weight: bold; }
p.important { border-left: 3px solid teal; }
a:hover { color: darkred; }
input:focus { outline: 2px solid #167d8d; }
```

Bien choisir id et class

Un id doit être unique dans le document. Une classe est réutilisable et un élément peut porter plusieurs classes. Pour des noms faciles à manipuler dans ce cours, utiliser des lettres minuscules, des chiffres après la première lettre et des tirets.

Les valeurs de class et de id sont sensibles à la casse : .important ne désigne pas class="Important". Les règles complètes des identifiants CSS autorisent davantage de caractères que cette convention simplifiée.

Sélecteurs combinés

```
.carte .titre { color: navy; } /* descendant */
.carte.titre { color: teal; } /* les deux classes */
li:first-child { font-weight: bold; }
tr:nth-child(even) { background-color: #eef4f7; }
```

Application

Créer trois cartes partageant la classe carte. Ajouter une classe selectionnee à une seule carte et lui donner une bordure différente.

Unités et modèle de boîte

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Unités absolues et relatives

• px : pixel CSS ; pt : point ; cm, mm et in : unités physiques.

• rem : taille de police de l’élément racine.

• em : taille de police de l’élément ; pour font-size, référence à la taille héritée.

• % : référence dépendant de la propriété, souvent une dimension du conteneur.

• vw et vh : centième de la largeur ou de la hauteur de la zone d’affichage.

• ch : mesure liée à la largeur du caractère zéro ; ex : hauteur d’x de la police.

16px est une taille courante par défaut, pas une égalité universelle pour 1em.

Les quatre zones d’une boîte

Du centre vers l’extérieur : contenu, padding, border, margin.

Avec box-sizing: content-box, width désigne la largeur du contenu. La largeur extérieure occupée, hors cas particuliers de mise en page, vaut :

Largeur = width + padding gauche/droit + border gauche/droit + margin gauche/droit.

```
.boite {
  width: 200px;
  padding: 10px;
  border: 2px solid teal;
  margin: 15px;
}
/* Boîte bordure comprise : 224px ; avec marges : 254px. */
```

Le modèle border-box

```
* { box-sizing: border-box; }
.boite {
  width: 200px;
  padding: 10px;
  border: 2px solid teal;
  margin: 15px;
}
/* Boîte bordure comprise : 200px ; avec marges : 230px. */
```

Notation courte

margin: 1em 0 2em 0.5em suit l’ordre haut, droite, bas, gauche. Deux valeurs définissent les côtés verticaux puis horizontaux. Une valeur s’applique aux quatre côtés. Les mêmes règles valent pour padding.

Attention

Certaines marges verticales de blocs peuvent fusionner. Pour une boîte en ligne non remplacée comme span, width et height ne s’appliquent pas comme sur un bloc ; utiliser inline-block si nécessaire.

Texte, couleurs, bordures et arrière-plans

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Texte et affichage

```
body {
  font-family: Arial, sans-serif;
  font-size: 1rem;
  line-height: 1.6;
  color: #16324f;
  background-color: #f4f7fa;
}
h1 { font-size: 2rem; text-align: center; }
.note { font-weight: bold; font-style: italic; }
.cache { display: none; }
.etiquette { display: inline-block; padding: 0.3rem; }
```

color règle la couleur du texte ; background-color celle du fond. Une couleur peut être un nom, une valeur hexadécimale, rgb() ou hsl(). opacity affecte l'élément entier et ses descendants ; une couleur avec alpha permet de ne rendre transparent que le fond.

Bordures et fonds

```
.carte {
  border: 2px solid #167d8d;
  border-radius: 8px;
  padding: 1rem;
  margin: 1rem 0;
  background: white;
}
.banniere {
  background: #16324f url("fond.png") no-repeat center;
  background-size: cover;
}
.degrade { background: linear-gradient(90deg, teal, navy); }
```

border-top, border-right, border-bottom et border-left ciblent un côté. border-width, border-style et border-color règlent séparément épaisseur, style et couleur. Exemples de styles : solid, dashed, dotted, double, inset.

background-repeat prend notamment repeat, no-repeat, repeat-x ou repeat-y ; background-attachment: fixed fixe l'image au viewport dans les situations usuelles ; background-position règle sa position. radial-gradient() crée un dégradé radial.

Application : modèle de boîte

Créer un div de largeur 400px, avec un padding supérieur de 40px, une bordure verte de 10px et une marge de 30px. Comparer sa largeur extérieure avec et sans border-box.

Vérification

Utiliser l'onglet « Éléments » du navigateur pour inspecter les styles calculés et le schéma du modèle de boîte.

Positionnement : relative et absolute

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Flux normal et propriété position

Dans le flux normal, les éléments occupent leur place selon leur type d’affichage. position: static est la valeur initiale. top, right, bottom et left servent à décaler des éléments positionnés.

Position relative

```
.etiquette {
  position: relative;
  top: 5px;
  left: 10px;
}
```

L’élément est décalé visuellement, mais sa place initiale reste réservée dans le flux. Il peut recouvrir un voisin sans que celui-ci se déplace.

Position absolute

```
<div class="carte">
  <img src="fleur.jpg" alt="Bouquet de fleurs">
  <span class="badge">Nouveau</span>
</div>

.carte { position: relative; width: 300px; }
.carte img { display: block; width: 100%; }
.badge {
  position: absolute;
  top: 10px;
  right: 10px;
  padding: 0.4rem;
  color: white;
  background: #147c89;
}
```

Un élément absolute est retiré du flux. Dans le cas courant illustré ici, il se place par rapport au plus proche ancêtre positionné, la carte. Sans cet ancêtre, sa référence peut être le bloc conteneur initial. Certaines propriétés comme transform peuvent aussi créer un bloc de référence.

Empilement

z-index permet de régler l’ordre d’empilement dans les contextes où il s’applique. Une valeur très élevée ne permet pas toujours de dépasser un élément appartenant à un autre contexte d’empilement.

Application

Afficher un prix dans le coin inférieur droit d’une image. Retirer position: relative de la carte et expliquer le changement. Éviter de positionner tous les blocs en absolute pour construire une page entière.

Positionnement : fixed et sticky

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

fixed : position généralement liée à la fenêtre

```
.aide {
  position: fixed;
  bottom: 1rem;
  right: 1rem;
  padding: 0.7rem;
  background: #16324f;
  color: white;
  z-index: 10;
}
```

L’élément est retiré du flux et reste généralement visible au défilement, relativement au viewport. Un ancêtre transformé peut modifier cette référence. Prévoir un espace suffisant pour ne pas cacher du contenu, notamment sur mobile.

sticky : ancrage pendant le défilement

```
.entete-table {
  position: sticky;
  top: 0;
  background: white;
  z-index: 2;
}
```

L’élément reste dans le flux. Il se comporte comme un élément normal jusqu’à atteindre le seuil fixé par top, puis reste ancré dans les limites de son contexte de défilement et de son conteneur. Sans seuil pertinent, l’effet attendu peut ne pas apparaître.

Comparaison

• static : flux normal, sans décalage par les propriétés de position.

• relative : place conservée, décalage visuel possible.

• absolute : hors flux, référence à un bloc conteneur.

• fixed : hors flux, référence généralement au viewport.

• sticky : place conservée, ancrage conditionné par le défilement.

Points de contrôle

Un ancêtre ayant overflow: auto ou hidden peut modifier le comportement de sticky. Une zone trop courte ne laisse pas assez d’espace pour observer l’ancrage. Vérifier aussi que le contenu reste accessible au clavier et à l’agrandissement du texte.

Application

Créer un en-tête sticky et un bouton d’aide fixed. Tester le défilement, le zoom et une fenêtre étroite. Vérifier qu’aucun champ de formulaire n’est caché par l’interface fixe.

Mise en page avec float et clear

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Flottants

float: left place un élément à gauche et permet au contenu suivant de l’entourer. float: right le place à droite. clear évite qu’un élément reste à côté de flottants antérieurs.

```
<header><img class="logo" src="logo.png" alt="Logo du lycée">
  <h1>Gestion des élèves</h1></header>
<div class="milieu">
  <nav><a href="inscription.html" target="cadre">Inscription</a></nav>
  <section class="contenu">
    <iframe name="cadre" title="Travail"
            src="inscription.html"></iframe>
  </section>
</div>
<footer>Lycée Rafèha</footer>

* { box-sizing: border-box; }
.logo { float: left; width: 70px; margin-right: 1rem; }
header { display: flow-root; }
nav { float: left; width: 180px; }
nav a { display: block; padding: 0.6rem; }
.contenu { margin-left: 200px; }
.milieu { display: flow-root; }
iframe { width: 100%; min-height: 500px; border: 0; }
footer { clear: both; text-align: center; }
```

Contenir les flottants

Un conteneur dont tous les enfants flottent peut ne pas les inclure dans sa hauteur ordinaire. display: flow-root crée un contexte de formatage qui contient les flottants. clear: both peut faire commencer le pied de page sous les flottants précédents.

Contraintes de l’activité

Les activités de mise en page du livre travaillent explicitement float et clear. Respecter cette consigne avant de proposer une variante avec flex.

Application

Construire une navigation de largeur fixe et une zone de contenu à droite. Ajouter une transition aux liens. Tester les débordements lorsque la fenêtre devient étroite ; proposer ensuite une règle adaptée aux petits écrans.

Flexbox, affichage et débordement

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Affichage

block occupe généralement une ligne disponible ; inline s’intègre au texte ; inline-block combine un placement en ligne et une boîte dimensionnable. display: none retire l’élément de l’affichage et de la mise en page ; visibility: hidden le masque en conservant sa place.

Conteneur flexible

```
<div class="cartes">
  <article>HTML5</article>
  <article>CSS3</article>
  <article>JavaScript</article>
</div>
```

```
.cartes {
  display: flex;
  flex-direction: row;
  flex-wrap: wrap;
  gap: 1rem;
  justify-content: space-between;
  align-items: stretch;
}
.cartes article {
  flex: 1 1 180px;
  padding: 1rem;
  border: 1px solid teal;
}
```

justify-content agit sur l’axe principal ; align-items sur l’axe transversal. Changer flex-direction change ces axes. gap ajoute un espace entre les éléments. flex-wrap autorise le retour à la ligne.

Débordement

```
.zone {
  max-height: 200px;
  overflow: auto;
}
.texte-long { overflow-wrap: anywhere; }
```

overflow: visible laisse déborder ; hidden masque le débordement ; auto ajoute un mécanisme de défilement si nécessaire ; scroll le prévoit même si le contenu ne déborde pas dans les situations usuelles. Vérifier que masquer un débordement ne rend pas une information inaccessible.

Application

Afficher trois cartes flexibles qui passent sur plusieurs lignes. Construire ensuite le même écran avec float pour comparer les mécanismes. L’emploi de Flexbox est un complément ; il ne remplace pas la consigne float d’un TP qui l’impose.

Liens, listes, tableaux et ombres

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

États des liens et focus

```
a:link { color: navy; }
a:visited { color: purple; }
a:hover { color: darkred; }
a:active { color: orange; }
a:focus-visible { outline: 3px solid teal; }
```

L'ordre link, visited, hover, active aide à éviter des conflits courants à spécificité égale. Le navigateur restreint les styles de visited pour protéger la confidentialité. Ne pas supprimer l'indicateur de focus sans solution de remplacement.

Listes

```
ul { list-style-type: square; list-style-position: outside; }
.menu { list-style: none; padding-left: 0; }
.illustrations { list-style-image: url("puce.png"); }
```

list-style combine type, position et image. Lorsqu'une image de puce valide est disponible, elle est utilisée à la place du type de puce.

Tableaux

```
table { width: 100%; border-collapse: collapse; }
th, td { border: 1px solid #9aabb8; padding: 0.5rem; }
th { background: #e8f3f5; text-align: left; }
tr:nth-child(even) { background: #f4f7fa; }
```

border-collapse: collapse fusionne les bordures voisines. table-layout: fixed fixe davantage le calcul des largeurs à partir de la table et de ses colonnes ; gérer les contenus longs.

Ombres et arrondis

```
.carte {
  border-radius: 12px 0 12px 0;
  box-shadow: 0 4px 12px rgb(0 0 0 / 20%);
}
h1 { text-shadow: 1px 2px 3px #b7c4cc; }
```

box-shadow : décalage horizontal, vertical, flou, puis éventuellement étalement et couleur. text-shadow s'applique au texte. Des valeurs multiples séparées par des virgules produisent plusieurs ombres. Conserver un contraste et une lisibilité suffisants.

Filtres et transformations 2D

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Filtres étudiés

```
.flou { filter: blur(3px); }
.gris { filter: grayscale(100%); }
.inverse { filter: invert(100%); }
.effet { filter: grayscale(100%) blur(1px); }
```

blur utilise une longueur positive ou nulle ; grayscale et invert admettent notamment un pourcentage de 0 % à 100 %. Leur valeur neutre est 0 ou 0 %. L’ordre des fonctions peut modifier le résultat. Le filtre s’applique au rendu de l’élément, y compris son contenu ; il ne modifie pas le fichier source.

Transformations

```
.rotation { transform: rotate(15deg); }
.taille { transform: scale(1.1); }
.decalage { transform: translate(20px, 10px); }
.inclinaison { transform: skew(10deg, 0deg); }
.combine {
  transform-origin: center;
  transform: translateX(15px) rotate(10deg) scale(1.05);
}
```

rotate tourne ; scale change l’échelle ; translate déplace ; skew incline. transform-origin fixe le point d’origine des transformations. Les fonctions se composent : modifier leur ordre peut changer l’effet observé.

Transform et mise en page

Une transformation visuelle ne redistribue pas le flux comme une modification des dimensions ordinaires. L’élément peut donc chevaucher un voisin. Un élément transformé peut aussi créer un nouveau contexte d’empilement et un bloc de référence pour certains descendants positionnés.

Survol progressif

```
.photo { transition: transform 0.3s, filter 0.3s; }
.photo:hover {
  transform: scale(1.05);
  filter: grayscale(100%);
}
```

Application

Créer une carte de fleur qui grandit légèrement au survol, puis tester blur, grayscale et invert séparément. Ajouter la même mise en évidence au focus si la carte est interactive ; ne pas réserver une information indispensable au seul survol.

Transitions et animations

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Transition entre deux états

Syntaxe simplifiée : propriété durée fonction-de-temporisation délai.

```
nav a {
  display: block;
  background-color: #edf5f6;
  transition: background-color 0.3s ease,
              transform 0.3s ease-out;
}
nav a:hover, nav a:focus-visible {
  background-color: #bfe5e9;
  transform: translateX(5px);
}
```

linear garde une progression régulière ; ease accélère puis ralentit ; ease-in privilégie un départ lent ; ease-out une fin lente ; ease-in-out combine les deux. Éviter transition: all si seules quelques propriétés doivent changer.

Animation avec étapes

```
@keyframes apparition {
  from { opacity: 0; transform: translateY(15px); }
  to { opacity: 1; transform: translateY(0); }
}
.carte {
  animation: apparition 0.6s ease-out 0s 1 normal both;
}
```

Les valeurs abrégées indiquent ici nom, durée, temporisation, délai, nombre d’itérations, sens et mode de remplissage. Des étapes 0 %, 50 % et 100 % permettent plusieurs états. animation-iteration-count: infinite répète sans fin ; éviter les mouvements permanents inutiles.

Réduire les mouvements

```
@media (prefers-reduced-motion: reduce) {
  .carte { animation: none; }
  nav a { transition: none; }
}
```

Application

Faire apparaître une carte une seule fois. Ajouter une transition de bordure aux champs ayant le focus. Vérifier le comportement lorsque l’utilisateur préfère moins d’animations. Les effets visuels ne doivent pas retarder la lecture ou masquer les erreurs de formulaire.

Transitions : durée, délai et progression

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Décomposer une transition

```
.bouton {
  background-color: #162330;
  color: white;
  transform: translateX(0);
  transition-property: background-color, transform;
  transition-duration: 0.3s, 0.5s;
  transition-timing-function: ease, ease-out;
  transition-delay: 0s, 0.1s;
}
.bouton:hover, .bouton:focus-visible {
  background-color: #516b78;
  transform: translateX(5px);
}
```

property : propriété animée ; duration : temps du changement ; timing-function : progression ; delay : attente avant le départ. Les listes séparées par des virgules associent leurs valeurs aux propriétés visées.

Notation courte équivalente

```
.bouton {
  transition: background-color 0.3s ease 0s,
              transform 0.5s ease-out 0.1s;
}
```

Dans une transition abrégée, la première durée est la durée de transition, la seconde le délai. Une durée nulle ne produit pas d’interpolation visible.

Comparer les progressions

linear : régulière ; ease : démarrage et fin progressifs ; ease-in : départ lent ; ease-out : arrivée lente ; ease-in-out : départ et arrivée lents. Ne pas confondre vitesse d’exécution d’un script et progression visuelle d’une propriété CSS.

Déclenchement et retour

Placer transition sur l’état de base pour que le passage vers le survol et le retour soient progressifs. Une transition nécessite un changement de valeur ; elle ne se répète pas seule comme une animation.

Application

Faire grandir un bouton au survol pendant 0,4 seconde, avec un délai de 0,1 seconde. Ajouter le même effet au focus et respecter prefers-reduced-motion. Éviter d’animer une propriété qui déplace continuellement les éléments pendant la lecture.

Animations : répétition, direction et remplissage

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Animation à plusieurs étapes

```
@keyframes signal {
  0%   { transform: translateX(0); opacity: 0.5; }
  50%  { transform: translateX(60px); opacity: 1; }
  100% { transform: translateX(0); opacity: 0.5; }
}
.indicateur {
  animation-name: signal;
  animation-duration: 2s;
  animation-delay: 0.5s;
  animation-iteration-count: 3;
  animation-direction: alternate;
  animation-timing-function: ease-in-out;
  animation-fill-mode: both;
  animation-play-state: running;
}
.indicateur:hover { animation-play-state: paused; }
```

Propriétés à retenir

iteration-count donne le nombre de répétitions ; infinite répète sans fin. direction : normal, reverse, alternate ou alternate-reverse. play-state permet running ou paused. Les animations peuvent fonctionner sans événement de survol.

Avant et après l’animation

fill-mode: none n’impose pas les états animés hors de leur période active ; backwards applique le premier état pertinent pendant le délai ; forwards conserve le dernier état pertinent après la fin ; both combine les deux. L’état pertinent dépend de direction et du nombre d’itérations.

Notation courte

```
.indicateur {
  animation: signal 2s ease-in-out 0.5s 3 alternate both;
}
@media (prefers-reduced-motion: reduce) {
  .indicateur { animation: none; }
}
```

Application

Reprendre une animation de largeur du support initial, puis comparer une animation de transform: scaleX. Expliquer leur effet sur le flux de la page. Créer trois variantes : une exécution unique, deux allers-retours et une animation suspendue au survol. Prévoir un arrêt accessible pour les mouvements prolongés.

Intégration, variables et types

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Rôle et chargement

Dans ce cours, JavaScript s’exécute dans le navigateur : il réagit aux événements, manipule le DOM et peut échanger avec un serveur. Il existe aussi des environnements JavaScript côté serveur.

```
<script src="script.js" defer></script>
<!-- Ou script interne placé à la fin de body -->
<script>
  console.log("Page prête");
</script>
```

Déclarer avec let

```
let nombre = 2;
let texte = "Bonjour";
let actif = true;
let resultat;                 // undefined
let selection = null;
nombre = nombre + 3;
console.log(nombre);          // 5
```

let déclare une variable dont la portée est le bloc délimité par des accolades. Il faut la déclarer avant de l’utiliser et ne pas la redéclarer dans le même bloc. Le cours utilise let pour les variables ; const reste possible pour une liaison qui ne sera pas réaffectée.

```
let message = "À l'extérieur";
if (actif) {
  let message = "Dans le bloc";
  console.log(message);
}
console.log(message);         // À l'extérieur
```

Types et identifiants

Les types usuels comprennent number, string, boolean, undefined, object et bigint ; null représente explicitement une absence de valeur. Les tableaux sont des objets. typeof null vaut "object", particularité historique du langage.

Un identifiant est sensible à la casse, ne commence pas par un chiffre et ne peut pas être un mot réservé. Préférer des noms explicites : note, total, nomEleve.

Commentaires et instructions

// commence un commentaire sur une ligne ; /* ... */ un commentaire de plusieurs lignes. Terminer les instructions par un point-virgule dans les exemples du cours.

Application

Déclarer le nom d’un élève, sa moyenne et son état d’admission, puis afficher leurs types avec typeof.

Opérateurs, conversions et dialogues

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Calcul et concaténation

```
let a = 7, b = 2;
console.log(a + b, a - b, a * b); // 9, 5, 14
console.log(a / b, a % b, a ** b); // 3.5, 1, 49
let prenom = "Amal";
let message = "Bonjour " + prenom;
console.log(message);
console.log(4 + "2");            // "42"
console.log(Number("4") + 2);     // 6
```

+=, -=, *= et /= combinent une opération et une affectation. ++ ajoute une unité ; -- soustrait une unité.

Dialogues du navigateur

alert(message) affiche un message. prompt(message) renvoie une chaîne ou null en cas d'annulation. confirm(message) renvoie true pour OK et false pour Annuler.

```
let saisie = prompt("Saisir une note de 0 à 20 :");
if (saisie !== null && saisie.trim() !== "") {
  let note = Number(saisie);
  if (Number.isFinite(note) && note >= 0 && note <= 20) {
    alert("Note : " + note);
  } else {
    alert("Note invalide.");
  }
}
```

Convertir correctement

• Number("12.5") produit 12.5 ; Number("12abc") produit NaN.

• Number("") produit 0 : vérifier une saisie vide avant conversion.

• Number.parseInt("12", 10) analyse un entier en base 10.

• Number.parseFloat("12.5abc") produit 12.5 : cette fonction ne valide pas toute la chaîne.

• String(42) produit "42" ; Number.isNaN(x) teste la valeur NaN sans conversion.

La base 10 explicite de parseInt rend l'intention claire. Sans base, l'analyse moderne utilise généralement 10, sauf notamment un préfixe 0x interprété en base 16.

Application

Demander deux nombres, gérer l'annulation et les chaînes vides, puis afficher leur somme après validation.

Conditions et choix multiples

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Comparaisons et logique

=== compare la valeur et le type ; !== teste leur différence. == et != peuvent convertir les opérandes : privilégier les comparaisons strictes. Les autres comparaisons sont <, <=, > et >=.

&& exprime ET ; || exprime OU ; ! exprime NON. Ces opérateurs utilisent des évaluations booléennes, mais && et || peuvent renvoyer un opérande non booléen.

```
let note = 14;
let resultat;
if (note < 0 || note > 20) {
  resultat = "Note invalide";
} else if (note >= 10) {
  resultat = "Admis";
} else {
  resultat = "Non admis";
}
console.log(resultat);
```

La variable resultat est déclarée avant les blocs, car elle est utilisée ensuite. Une variable déclarée avec let à l'intérieur d'un seul bloc ne serait pas accessible à cet endroit.

switch

```
let fruit = "Pommes";
switch (fruit) {
  case "Oranges":
    console.log("2 DT le kilo");
    break;
  case "Pommes":
    console.log("3,5 DT le kilo");
    break;
  case "Bananes":
  case "Cerises":
    console.log("9,5 DT le kilo");
    break;
  default:
    console.log("Fruit non disponible");
}
```

switch compare strictement l'expression aux valeurs des case. break évite de poursuivre dans le cas suivant.

Opérateur conditionnel

let etat = note >= 10 ? "Admis" : "Non admis";

Cette expression choisit l'une des deux valeurs ; la réserver à des choix simples pour garder un code lisible.

Boucles et accumulation

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

for : nombre d’itérations connu

```
let somme = 0;
for (let i = 1; i <= 10; i++) {
  somme += i;
}
console.log(somme); // 55 ; i n'est pas accessible ici.
```

La condition du for est une condition de poursuite : le bloc s’exécute tant qu’elle est vraie.

while : test avant l’exécution

```
let nombre = 1;
while (nombre < 10) {
  nombre++;
}
console.log(nombre); // 10
```

do...while : au moins une exécution

```
let total = 0;
let continuer = true;
do {
  let saisie = prompt("Nombre positif ; négatif pour finir :");
  if (saisie === null) { break; }
  let nombre = Number(saisie);
  if (saisie.trim() === "" || !Number.isFinite(nombre)) {
    alert("Saisie invalide");
    continue;
  }
  continuer = nombre >= 0;
  if (continuer) { total += nombre; }
} while (continuer);
alert("Somme : " + total);
```

continue passe à l’itération suivante ; break quitte la boucle. continuer est initialisé à true : une saisie invalide fait recommencer la boucle sans modifier le total. Le nombre négatif de fin n’est pas ajouté à la somme.

Compter des prénoms

```
let prenoms = [], prenom;
while ((prenom = prompt("Prénom ; vide pour terminer :")) !== null) {
  if (prenom.trim() === "") { break; }
  prenoms.push(prenom.trim());
}
alert(prenoms.length + " prénoms : " + prenoms.join(", "));
```

Vigilance

Prévoir une progression et une condition d’arrêt. Une boucle synchrone infinie bloque l’interface du navigateur.

TP : jeu de devinettes

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Objectif

Générer un entier entre 1 et 12 puis permettre à l'utilisateur de le deviner. Compter seulement les propositions valides et autoriser l'annulation.

```
let minimum = 1;
let maximum = 12;
let secret = Math.floor(Math.random() *
             (maximum - minimum + 1)) + minimum;
let essais = 0;
let trouve = false;
let indice = "";
while (!trouve) {
  let saisie = prompt("Entier de 1 à 12. " + indice);
  if (saisie === null) {
    alert("Partie annulée.");
    break;
  }
  let nombre = Number(saisie);
  if (saisie.trim() === "" || !Number.isInteger(nombre)
      || nombre < minimum || nombre > maximum) {
    indice = "La proposition doit être un entier valide.";
    continue;
  }
  essais++;
  if (nombre > secret) {
    indice = "Cherchez plus petit.";
  } else if (nombre < secret) {
    indice = "Cherchez plus grand.";
  } else {
    trouve = true;
    alert("Bravo ! Trouvé en " + essais + " essai(s).");
  }
}
```

Comprendre le tirage

Math.random() renvoie un réel dans [0, 1[. Math.floor(x) renvoie le plus grand entier inférieur ou égal à x. Le calcul multiplie par le nombre de valeurs possibles, puis ajoute la borne minimale.

Questions

1. Pourquoi ne faut-il pas utiliser Math.round() dans cette formule ?

2. À quel endroit la portée de la variable nombre se termine-t-elle ?

3. Que devient essais lorsque l'utilisateur saisit une lettre ?

Améliorations

Limiter le nombre d'essais, proposer une nouvelle partie et remplacer prompt par un formulaire HTML. Math.random() ne convient pas à la génération de secrets de sécurité.

SCANNER LA FICHE
ou cliquer sur le QR code

DOM : sélectionner et modifier des éléments

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Le DOM

Le Document Object Model représente le document sous forme de nœuds et d'objets. JavaScript peut sélectionner des éléments, lire leurs propriétés et modifier leur contenu.

• document.getElementById("intro") : élément d'identifiant intro, ou null.

• document.querySelector(".carte") : premier élément correspondant, ou null.

• document.querySelectorAll(".carte") : NodeList de tous les éléments correspondants.

• getElementsByTagName("p") et getElementsByClassName("carte") : collections dynamiques.

• getElementsByName("groupe") : collection des éléments portant ce name.

```
<p id="intro">Un texte initial.</p>
<img id="photo" src="avant.png" alt="Version initiale">

let paragraphe = document.getElementById("intro");
if (paragraphe !== null) {
  console.log(paragraphe.textContent);
  paragraphe.textContent = "Un autre texte";
  paragraphe.classList.add("important");
}
let photo = document.querySelector("#photo");
photo.setAttribute("src", "apres.png");
photo.alt = "Nouvelle version";
```

textContent ou innerHTML ?

textContent traite la valeur comme du texte. innerHTML lit ou interprète du balisage HTML. Ne pas affecter à innerHTML une saisie utilisateur non maîtrisée : risque d'injection de code (XSS). Pour afficher du texte, préférer textContent.

Autres accès

```
let formulaire = document.forms["inscription"];
let premiereImage = document.images[0];
let cartes = document.querySelectorAll(".carte");
for (let carte of cartes) {
  carte.classList.add("visible");
}
```

À retenir

Une collection DOM n'est pas nécessairement un tableau Array. Les noms des propriétés sont sensibles à la casse. Charger le script avec defer pour que les éléments HTML existent lors de leur sélection.

DOM : lire et contrôler un formulaire

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Propriétés essentielles

• input.value, textarea.value et select.value : valeur textuelle du champ.

• checkbox.checked et radio.checked : état booléen.

• select.options : collection des options ; options.length : taille.

• select.selectedIndex : indice de l’option sélectionnée, ou -1.

• option.selected, option.text et option.value : état, libellé et valeur.

```
<form id="calcul">
  <label for="note">Note</label>
  <input id="note" name="note" type="number"
         min="0" max="20" step="0.5" required>
  <label><input id="accord" type="checkbox" required>
    Je confirme la saisie</label>
  <button type="submit">Vérifier</button>
</form>
<p id="resultat"></p>

let formulaire = document.getElementById("calcul");
formulaire.addEventListener("submit", function (event) {
  event.preventDefault();
  let champ = document.getElementById("note");
  let accord = document.getElementById("accord").checked;
  let note = Number(champ.value);
  let sortie = document.getElementById("resultat");
  if (champ.value === "" || !Number.isFinite(note)
      || note < 0 || note > 20 || !accord) {
    sortie.textContent = "Vérifier les données.";
    return;
  }
  sortie.textContent = note >= 10 ? "Admis" : "Non admis";
});
```

Modifier une liste existante

```
let liste = document.getElementById("classe");
liste.add(new Option("4ème SI 3", "4SI3"));
if (liste.options.length > 0) { liste.remove(0); }
```

Pour les radios, document.querySelector('input[name="groupe"]:checked') renvoie le bouton sélectionné, ou null. Tester null avant d’accéder à sa valeur. Le contrôle serveur reste indispensable.

Fonctions, portée et événements

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Définir et appeler une fonction

```
function doubler(nombre) {
  let resultat = nombre * 2;
  return resultat;
}
let valeur = doubler(7);
console.log(valeur); // 14
```

Un paramètre est un nom dans la définition ; un argument est une valeur transmise lors de l'appel. return termine la fonction et renvoie une valeur. Sans retour explicite, le résultat est undefined.

Portée

```
let message = "Message extérieur";
function afficher() {
  let message = "Message local";
  console.log(message);
}
afficher();
console.log(message); // Message extérieur
```

Les arguments sont transmis par valeur. Pour un objet, la valeur transmise est une référence : modifier ses propriétés peut modifier l'objet partagé, mais réaffecter le paramètre ne réaffecte pas la variable de l'appelant.

Associer un événement

```
<button id="bouton" type="button">Cliquer</button>
<p id="sortie"></p>

function reagir(event) {
  document.getElementById("sortie").textContent = "Bonjour";
  console.log(event.currentTarget.id);
}
let bouton = document.getElementById("bouton");
bouton.addEventListener("click", reagir);
// Pour retirer cet écouteur :
// bouton.removeEventListener("click", reagir);
```

Méthodes et événements usuels

onclick = reagir associe un gestionnaire, remplacé par une affectation ultérieure. addEventListener autorise plusieurs écouteurs et est recommandé. Ne pas écrire reagir() lors de l'association : cela appellerait immédiatement la fonction.

Événements courants : click, input, change, submit, focus et blur. event.target désigne la cible initiale ; event.currentTarget l'élément portant l'écouteur. Éviter les chaînes de code dans les attributs d'événements.

Nombres, chaînes et méthodes usuelles

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Nombres et Math

```
let nombre = Number("123");
console.log(nombre.toString());   // "123"
console.log((6).toString(2));      // "110"
console.log(Number.isNaN(NaN));    // true
console.log(Math.floor(4.8));      // 4
console.log(Math.round(4.8));      // 5
console.log(Math.abs(-7));        // 7
```

Number utilise généralement une représentation IEEE 754 en double précision. Certains décimaux ne sont pas représentés exactement. Éviter new Number() pour un simple nombre : il crée un objet dont valueOf() extrait la valeur primitive.

Chaînes

```
let texte = "  Bonjour Tunis  ";
let propre = texte.trim();
console.log(propre.length);           // 13
console.log(propre.charAt(0));        // "B"
console.log(propre.indexOf("Tunis")); // 8
console.log(propre.lastIndexOf("o")); // 4
console.log(propre.substring(0, 7));  // "Bonjour"
console.log(propre.slice(-5));        // "Tunis"
console.log(propre.toUpperCase());    // "BONJOUR TUNIS"
console.log(propre.toLowerCase());    // "bonjour tunis"
console.log("Bonjour ".concat("Amal")); // "Bonjour Amal"
console.log(propre.split(" "));       // ["Bonjour", "Tunis"]
```

indexOf et lastIndexOf renvoient -1 en l’absence de correspondance. substring utilise un indice de fin exclu et échange les bornes si nécessaire. slice accepte des indices négatifs. L’ancienne méthode substr est à éviter dans de nouveaux exemples.

length mesure des unités de code UTF-16, pas toujours des caractères visibles : certains symboles comptent pour deux unités ou davantage. Les chaînes sont immuables : les méthodes renvoient de nouvelles valeurs.

Pourquoi éviter eval ?

eval exécute une chaîne comme du code. Il ne faut pas l’utiliser pour calculer une expression saisie par l’utilisateur. Employer les opérateurs et les conversions explicites.

Application

Nettoyer un nom avec trim, le convertir en majuscules et vérifier qu’il contient au moins deux caractères.

Tableaux et parcours

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Créer, lire et modifier un tableau

```
let fruits = ["Pomme", "Banane"];
console.log(fruits[0]);                 // Pomme
console.log(fruits.length);             // 2
fruits.push("Orange");                  // ajout à la fin
let dernier = fruits.pop();             // retire Orange
fruits.unshift("Fraise");               // ajout au début
let premier = fruits.shift();           // retire Fraise
console.log(fruits.join(", "));         // Pomme, Banane
console.log(fruits.toString());         // Pomme,Banane
```

Les indices commencent à zéro. Pour un tableau dense, le dernier indice est length - 1. Un indice absent renvoie undefined. Dans un tableau creux, length peut dépasser le nombre d’éléments réellement présents.

Parcourir

```
for (let i = 0; i < fruits.length; i++) {
  console.log(i, fruits[i]);
}
for (let fruit of fruits) {
  console.log(fruit);
}
fruits.forEach(function (fruit, indice) {
  console.log(indice, fruit);
});
```

for...of parcourt les valeurs. forEach appelle une fonction pour chaque élément présent ; break ne permet pas de quitter ce parcours. Utiliser une boucle classique si un arrêt anticipé est nécessaire.

Exemple de traitement

```
let notes = [12, 15, 9, 16];
let somme = 0;
for (let note of notes) { somme += note; }
let moyenne = notes.length > 0 ? somme / notes.length : null;
console.log(moyenne); // 13
notes.sort(function (a, b) { return a - b; });
console.log(notes);   // [9, 12, 15, 16]
```

sort sans fonction compare les représentations textuelles. Pour trier des nombres dans l’ordre croissant, fournir la fonction de comparaison a - b.

Application

Afficher la plus grande note et compter les notes supérieures ou égales à 10. Prévoir le cas d’un tableau vide.

Dates, temporisation et horloge numérique

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Objet Date

```
let maintenant = new Date();
let exemple = new Date(2026, 9, 21, 12, 30, 0);
console.log(exemple.getFullYear()); // 2026
console.log(exemple.getMonth());    // 9 : octobre
console.log(exemple.getDate());     // 21 : jour du mois
```

getDay renvoie le jour de la semaine de 0 (dimanche) à 6. getHours, getMinutes, getSeconds et getMilliseconds renvoient respectivement 0–23, 0–59, 0–59 et 0–999. Ces méthodes lisent l'heure locale du navigateur.

getTime() renvoie les millisecondes depuis le 1er janvier 1970 à 00:00:00 UTC ; setTime(ms) modifie la date. Un timestamp Unix classique est exprimé en secondes : ne pas confondre les unités.

TP : horloge

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Horloge</title>
</head>
<body>
  <span id="horloge" aria-label="Heure actuelle"></span>
  <script>
    function afficherHorloge() {
      let date = new Date();
      let h = String(date.getHours()).padStart(2, "0");
      let m = String(date.getMinutes()).padStart(2, "0");
      let s = String(date.getSeconds()).padStart(2, "0");
      document.getElementById("horloge").textContent =
        h + ":" + m + ":" + s;
    }
    afficherHorloge();
    let minuterie = setInterval(afficherHorloge, 1000);
    // clearInterval(minuterie) permet l'arrêt.
  </script>
</body>
</html>
```

setTimeout(fonction, délai) prévoit un appel unique ; setInterval(fonction, délai) prévoit des appels répétés. Le délai est en millisecondes et n'est pas une garantie d'exécution exacte. Passer une fonction plutôt qu'une chaîne de code.

Compléments : Math, chaînes et sorties

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Math et nombres

```
console.log(Math.ceil(4.2));       // 5
console.log(Math.trunc(-4.8));     // -4
console.log(Math.sqrt(81));        // 9
console.log(Math.pow(2, 3));       // 8
console.log(Math.min(5, 2, 9));    // 2
console.log(Math.max(5, 2, 9));    // 9
console.log((12.345).toFixed(2));  // "12.35", une chaîne
console.log(isNaN("abc"));       // true : conversion préalable
console.log(Number.isNaN("abc")); // false : pas de conversion
```

Math.floor(-4.8) vaut -5, tandis que Math.trunc(-4.8) vaut -4. Les opérations décimales restent sujettes à la représentation binaire des nombres.

Remplacer et rechercher

```
let texte = "HTML et HTML";
console.log(texte.replace("HTML", "CSS")); // CSS et HTML
console.log(texte.replaceAll("HTML", "CSS")); // CSS et CSS
console.log(texte.includes("HTML")); // true
console.log("abc".startsWith("a")); // true
```

Les méthodes renvoient de nouvelles chaînes. La comparaison est généralement sensible à la casse.

Sorties : choisir le bon outil

console.log facilite le débogage. alert interrompt l’interaction jusqu’à sa fermeture. textContent affiche du texte ; innerHTML interprète du HTML. document.write peut remplacer le document lorsqu’il est appelé après son chargement : éviter son emploi dans les nouveaux exemples.

Ordre des opérations

```
console.log(1 + 2 + "3"); // "33"
console.log("1" + 2 + 3); // "123"
console.log("Total : " + (2 + 3)); // "Total : 5"
```

Application

Afficher une moyenne arrondie à deux décimales sans confondre une valeur numérique et sa représentation textuelle. Vérifier une saisie avec Number.isFinite après conversion explicite.

Médias, styles et dates de formulaire

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Contrôler un média existant

```
<video id="film" controls src="demo.mp4"></video>
<button id="lecture" type="button">Lire</button>
<button id="pause" type="button">Pause</button>
<p id="message"></p>

let film = document.getElementById("film");
document.getElementById("lecture").addEventListener("click",
  function () {
    film.play().catch(function () {
      document.getElementById("message").textContent =
        "Lecture impossible : vérifier le fichier et les autorisations.";
    });
  });
document.getElementById("pause").addEventListener("click",
  function () { film.pause(); });
```

play() renvoie une promesse : la lecture peut être refusée par le navigateur ou échouer. Un clic utilisateur est préférable à un démarrage automatique non demandé.

Modifier la présentation

```
let element = document.getElementById("message");
element.style.backgroundColor = "#edf5f6";
element.classList.toggle("important");
```

Une propriété CSS avec tiret devient généralement une propriété JavaScript en camelCase. Préférer classList pour des ensembles de styles réutilisables.

Date non future

```
function dateNonFuture(valeur) {
  if (!/^\d{4}-\d{2}-\d{2}$/.test(valeur)) { return false; }
  let morceaux = valeur.split("-").map(Number);
  let a = morceaux[0], m = morceaux[1], j = morceaux[2];
  let date = new Date(a, m - 1, j);
  let limite = new Date();
  limite.setHours(0, 0, 0, 0);
  return date.getFullYear() === a && date.getMonth() === m - 1
    && date.getDate() === j && date <= limite;
}
```

Le constructeur normalise les dépassements : comparer les composantes permet de rejeter une date comme le 31 février. Vérifier la même règle sur le serveur, dont le fuseau peut différer de celui du navigateur.

Notions relationnelles et contraintes

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Vocabulaire

Une base de données est un ensemble organisé et persistant de données. Un SGBD permet de définir les structures, manipuler les données, gérer les accès concurrents et contrôler leur intégrité et leur sécurité. MySQL est un SGBD relationnel ; SQL est le langage utilisé pour dialoguer avec lui.

• Entité : objet du monde réel, par exemple un élève ou une classe.

• Attribut : propriété d’une entité, par exemple son nom.

• Table : ensemble de lignes décrites par des colonnes.

• Ligne, enregistrement ou n-uplet : occurrence d’une entité ou d’une association.

• Colonne ou champ : propriété ayant un type et des contraintes.

Clés et intégrité

Une clé primaire identifie chaque ligne ; elle est unique et non nulle. Elle peut être composée de plusieurs colonnes. Une clé étrangère référence une clé d’une autre table et assure la cohérence des liens.

NOT NULL impose une valeur ; UNIQUE impose l’unicité ; CHECK impose une condition ; FOREIGN KEY définit une référence. Une colonne nullable peut contenir NULL, qui signifie une information absente ou inconnue, et non une chaîne vide ou zéro.

Relation un à plusieurs

CLASSE(id_classe, nom_classe)

ELEVE(num_inscription, nom, prenom, date_naissance, adresse, id_classe#)

Une classe contient plusieurs élèves ; chaque élève appartient à une classe. La clé id_classe est ajoutée à ELEVE comme clé étrangère. Le symbole # est une convention de description, pas une syntaxe de nom de colonne SQL.

Relation plusieurs à plusieurs

ATHLETE(anum, anom, aage)

COURSE(cnum, cnom)

PARTICIPATION(cnum#, anum#)

Un athlète participe à plusieurs courses et une course accueille plusieurs athlètes. PARTICIPATION porte deux clés étrangères et une seule clé primaire composée de (cnum, anum).

Intérêts d’une base

Centraliser les informations, limiter les redondances inutiles, partager les données et assurer leur cohérence. La confidentialité dépend des droits d’accès ; elle n’est pas automatiquement garantie par le seul stockage en base.

Application

Identifier les clés primaires et étrangères d’un système de location de véhicules. Expliquer pourquoi un nom de client ne constitue pas une bonne clé primaire.

SQL : commandes et types MySQL

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Familles de commandes

• Définition des données : CREATE, ALTER, DROP.

• Manipulation des données : INSERT, UPDATE, DELETE ; interrogation : SELECT.

• Contrôle des accès : GRANT et REVOKE, étudiés ici uniquement comme repères.

Les exemples suivants ciblent MySQL 8.0.16 ou ultérieur, avec le moteur InnoDB. À partir de cette version, les contraintes CHECK sont effectivement appliquées. Des différences peuvent exister avec MariaDB et d'autres SGBD.

```
CREATE DATABASE gestion_location
  CHARACTER SET utf8mb4;
USE gestion_location;
SHOW DATABASES;
SHOW TABLES;
```

Types numériques

• TINYINT : entier signé de -128 à 127 ; UNSIGNED de 0 à 255.

• INT : entier signé de -2147483648 à 2147483647 ; UNSIGNED de 0 à 4294967295.

• DECIMAL(p,s) : décimal exact, p chiffres au total dont s après la virgule. DECIMAL(8,3) autorise cinq chiffres avant et trois après le point.

• NUMERIC est un synonyme de DECIMAL dans MySQL.

• FLOAT et DOUBLE sont approximatifs : préférer DECIMAL pour les montants monétaires.

Types textuels et binaires

• CHAR(n) : chaîne de longueur fixe ; VARCHAR(n) : longueur variable maximale n, limitée aussi par l'encodage et la taille maximale d'une ligne.

• TEXT : texte ; BLOB : données binaires. Leur taille est limitée en octets, et non toujours à 65535 caractères.

• La sensibilité à la casse d'un texte dépend de sa collation ; TEXT ne signifie pas « ASCII ».

• Téléphone et CIN sont des chaînes : un zéro initial doit être conservé.

Types temporels

DATE : 2026-10-21 ; TIME : 12:30:00 ; DATETIME : 2026-10-21 12:30:00 ; YEAR : 2026.

TIMESTAMP applique des conversions liées au fuseau horaire de la session ; DATETIME stocke une date et une heure civiles sans cette conversion automatique.

Conventions

Utiliser des identifiants sans espaces et conserver exactement la même casse pour les noms de tables. Les dates littérales sont écrites 'AAAA-MM-JJ'. Les chaînes utilisent des apostrophes droites.

Création des tables : gestion des locations

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Schéma cohérent

Créer les tables parentes avant la table qui les référence. La clé de location inclut la date pour permettre plusieurs locations d'un même véhicule par un même client à des dates différentes.

```
CREATE TABLE client (
  ncin CHAR(8) PRIMARY KEY,
  nom VARCHAR(50) NOT NULL,
  prenom VARCHAR(50) NOT NULL,
  adresse VARCHAR(100),
  ville VARCHAR(50) DEFAULT 'Tunis'
) ENGINE=InnoDB;

CREATE TABLE vehicule (
  immat VARCHAR(20) PRIMARY KEY,
  marque VARCHAR(30) NOT NULL,
  modele VARCHAR(30) NOT NULL,
  annee_acq YEAR NOT NULL,
  CONSTRAINT ck_annee CHECK (annee_acq >= 1990)
) ENGINE=InnoDB;

CREATE TABLE location (
  immat VARCHAR(20) NOT NULL,
  ncin CHAR(8) NOT NULL,
  date_loc DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  duree_loc INT NOT NULL,
  cout_loc DECIMAL(8,3) NOT NULL,
  PRIMARY KEY (immat, ncin, date_loc),
  CONSTRAINT fk_location_vehicule FOREIGN KEY (immat)
    REFERENCES vehicule(immat) ON DELETE RESTRICT,
  CONSTRAINT fk_location_client FOREIGN KEY (ncin)
    REFERENCES client(ncin) ON DELETE RESTRICT,
  CONSTRAINT ck_duree CHECK (duree_loc > 0),
  CONSTRAINT ck_cout CHECK (cout_loc > 0)
) ENGINE=InnoDB;
```

Intégrité référentielle

RESTRICT refuse la suppression d'un parent encore référencé. CASCADE supprime automatiquement ses enfants : à choisir seulement si cela correspond à la règle de gestion. Pour conserver un historique de locations, RESTRICT est plus prudent.

Dates et contraintes

Ne pas écrire CHECK(date_loc <= NOW()) : MySQL n'autorise pas cette fonction non déterministe dans un CHECK. Vérifier cette règle dans le traitement applicatif ou un déclencheur adapté. DEFAULT CURRENT_TIMESTAMP n'est pas réservé au seul type TIMESTAMP ; il est aussi utilisable avec DATETIME.

Modifier et supprimer une structure

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Exemples indépendants à tester sur une copie

```
ALTER TABLE client ADD COLUMN telephone VARCHAR(8);
ALTER TABLE client MODIFY COLUMN telephone VARCHAR(20);
ALTER TABLE client ALTER COLUMN ville SET DEFAULT 'Tunis';
ALTER TABLE client DROP COLUMN adresse;
ALTER TABLE vehicule ADD CONSTRAINT ck_annee_max
  CHECK (annee_acq <= 2026);
ALTER TABLE vehicule DROP CHECK ck_annee_max;
DESCRIBE client;
SHOW CREATE TABLE location;
```

Un numéro de téléphone reste textuel. Lors d’un MODIFY COLUMN, préciser les propriétés à conserver, notamment NOT NULL et DEFAULT, car certaines peuvent être perdues si on les omet.

Clé primaire composée

Pour une table location ancienne dont la clé était seulement (immat, ncin), la correction est :

```
ALTER TABLE location
  DROP PRIMARY KEY,
  ADD PRIMARY KEY (immat, ncin, date_loc);
```

La virgule sépare les deux modifications. Dans le schéma de la fiche précédente, la clé est déjà correcte : il n’est pas nécessaire de la modifier.

Contraintes et renommage

```
ALTER TABLE location DROP FOREIGN KEY fk_location_client;
ALTER TABLE vehicule RENAME TO voiture;
```

Ne pas supprimer une clé primaire référencée sans analyser les clés étrangères et les index associés. DISABLE KEYS et ENABLE KEYS ne désactivent pas les contraintes de clé primaire ou de clé étrangère InnoDB ; ce ne sont pas des commandes de suspension de l’intégrité.

Suppression : prudence

```
-- À exécuter seulement sur une base de test sauvegardée.
DROP TABLE location;
DROP TABLE vehicule;
DROP TABLE client;
-- DROP DATABASE gestion_location;
```

Supprimer les tables enfants avant les parentes. Ces commandes supposent que vehicule n’a pas été renommée. DROP détruit une structure ; DELETE supprime des lignes en conservant la table.

Application

Ajouter une colonne email au client, puis vérifier la nouvelle structure. Ne pas exécuter toutes les commandes de cette fiche à la suite sur la base servant aux autres exercices.

Insérer, modifier et supprimer des lignes

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

INSERT : préciser les colonnes

```
INSERT INTO client (ncin, nom, prenom, ville)
VALUES ('01234567', 'Tounsi', 'Wael', 'Tunis');
INSERT INTO vehicule (immat, marque, modele, annee_acq)
VALUES ('123TU4567', 'Renault', 'Clio', 2026);
INSERT INTO location (immat, ncin, date_loc, duree_loc, cout_loc)
VALUES ('123TU4567', '01234567', '2026-10-21 09:00:00',
        3, 180.000);
```

Les valeurs doivent respecter les types, NOT NULL, l'unicité, les CHECK et les clés étrangères. Insérer les parents avant les enfants. Une chaîne numérique comme la CIN garde ses apostrophes pour préserver sa forme.

UPDATE : attention à WHERE

```
UPDATE client
SET ville = 'Sfax', telephone = '20123456'
WHERE ncin = '01234567';

UPDATE location
SET cout_loc = ROUND(cout_loc * 1.18, 3)
WHERE date_loc >= '2026-10-01'
  AND date_loc < '2026-11-01';
```

Le premier exemple suppose la colonne telephone ajoutée dans la fiche précédente. Le second augmente de 18 % le coût des locations d'octobre 2026. Sans WHERE, UPDATE modifierait toutes les lignes.

DELETE

```
DELETE FROM location
WHERE immat = '123TU4567'
  AND ncin = '01234567'
  AND date_loc = '2026-10-21 09:00:00';
```

Sans WHERE, DELETE FROM location supprime toutes les lignes de location. La suppression d'un parent référencé est refusée avec RESTRICT, ou propagée avec CASCADE si cette action a été définie.

Méthode de travail

1. Écrire un SELECT avec la même condition pour vérifier les lignes visées.
2. Vérifier la sauvegarde et, si nécessaire, utiliser une transaction.
3. Exécuter la modification et contrôler le nombre de lignes affectées.

Exercice

Changer la durée d'une location précisément identifiée, puis supprimer uniquement cette location. Expliquer pourquoi une condition portant seulement sur la CIN peut viser plusieurs lignes.

Base école : tables et données des exercices

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Un jeu d’essai commun aux sélections simples

La relation entre une classe et ses élèves est modélisée par une clé étrangère. Les recherches du chapitre interrogent toujours une seule table ; aucune jointure n’est requise.

```
CREATE DATABASE ecole_3si CHARACTER SET utf8mb4;
USE ecole_3si;
CREATE TABLE classe (
  id_classe INT PRIMARY KEY,
  nom_classe VARCHAR(20) NOT NULL UNIQUE
) ENGINE=InnoDB;
CREATE TABLE eleve (
  num_eleve INT PRIMARY KEY,
  nom VARCHAR(50) NOT NULL,
  prenom VARCHAR(50) NOT NULL,
  date_naissance DATE NOT NULL,
  ville VARCHAR(50),
  moyenne DECIMAL(4,2) NOT NULL CHECK (moyenne BETWEEN 0 AND 20),
  id_classe INT NOT NULL,
  FOREIGN KEY (id_classe) REFERENCES classe(id_classe)
) ENGINE=InnoDB;
INSERT INTO classe VALUES (1,'3SI1'), (2,'3SI2');
INSERT INTO eleve VALUES
(1,'Benali','Amine','2009-03-14','Tunis',15.50,1),
(2,'Trabelsi','Sara','2009-07-02','Sfax',17.25,1),
(3,'Gharbi','Yassine','2008-11-25','Tunis',9.50,2),
(4,'Haddad','Leila','2009-01-19',NULL,12.00,2),
(5,'Ayadi','Nour','2009-05-30','Sousse',14.75,1);
```

Lire le schéma

classe est la table parente ; eleve la table fille. Chaque élève doit référencer une classe existante. La valeur NULL de ville signifie une information absente, pas une chaîne vide.

Mettre à jour avec prudence

```
UPDATE eleve SET moyenne = 10.00 WHERE num_eleve = 3;
DELETE FROM eleve WHERE num_eleve = 5;
```

Ces modifications sont des exercices indépendants : réimporter le jeu initial avant de comparer les réponses des fiches suivantes. Ne pas exécuter les suppressions sur une base réelle.

SELECT simple : choisir les colonnes

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Principe et limite de ce chapitre

Une requête de sélection simple lit une seule table. Elle choisit des colonnes et, si nécessaire, des lignes. Le livre s’arrête à ce niveau, avec filtres et tri.

```
USE ecole_3si;
SELECT * FROM eleve;
SELECT nom, prenom FROM eleve;
SELECT num_eleve, moyenne FROM eleve;
SELECT nom, moyenne AS note_moyenne FROM eleve;
```

* désigne toutes les colonnes. La liste explicite des colonnes rend la requête plus claire. AS nomme une colonne du résultat sans renommer la colonne stockée. Un SELECT ne modifie pas les lignes de la table.

Projection et sélection

La projection choisit les colonnes du résultat. La sélection, avec WHERE, choisit les lignes. Les deux opérations peuvent être combinées.

```
SELECT nom, prenom, moyenne
FROM eleve
WHERE moyenne >= 10;
```

DISTINCT

```
SELECT DISTINCT ville FROM eleve;
SELECT DISTINCT nom, prenom FROM eleve;
```

DISTINCT élimine les doublons du résultat selon l’ensemble des colonnes sélectionnées. Il ne supprime aucune ligne de la table. Deux élèves peuvent porter le même nom sans être le même enregistrement.

Résultat attendu sur le jeu initial

SELECT nom, prenom FROM eleve produit cinq lignes et deux colonnes. Le filtre moyenne >= 10 conserve quatre élèves. Sans ORDER BY, l’ordre de retour n’est pas garanti.

Application

Écrire les requêtes qui affichent tous les élèves, leurs noms et dates de naissance, puis seulement les élèves de la classe d’identifiant 1. Décrire le nombre de colonnes du résultat sans utiliser une fonction de calcul SQL.

WHERE : comparaisons, logique et valeurs absentes

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Comparer les valeurs

```
SELECT nom, prenom FROM eleve WHERE moyenne >= 15;
SELECT * FROM eleve WHERE id_classe = 1;
SELECT nom, ville FROM eleve WHERE ville <> 'Tunis';
SELECT nom, date_naissance FROM eleve
WHERE date_naissance < '2009-01-01';
```

Les opérateurs sont =, <>, !=, <, <=, > et >=. Les chaînes et les dates littérales utilisent des apostrophes droites ; les dates sont écrites au format AAAA-MM-JJ.

AND, OR et NOT

```
SELECT nom, moyenne FROM eleve
WHERE id_classe = 1 AND moyenne >= 14;
SELECT nom, ville FROM eleve
WHERE ville = 'Tunis' OR ville = 'Sfax';
SELECT nom, moyenne FROM eleve
WHERE NOT (moyenne < 10);
SELECT nom, ville, moyenne FROM eleve
WHERE (ville = 'Tunis' OR ville = 'Sfax') AND moyenne >= 15;
```

AND est prioritaire sur OR. Les parenthèses explicitent le regroupement et évitent d’attribuer un filtre à une seule branche du OU alors qu’il devait s’appliquer aux deux.

NULL

```
SELECT nom, prenom FROM eleve WHERE ville IS NULL;
SELECT nom, prenom FROM eleve WHERE ville IS NOT NULL;
```

Ne pas utiliser = NULL ou <> NULL. Une comparaison ordinaire à une valeur inconnue n’est pas vraie : ville <> 'Tunis' ne conserve pas la ligne dont ville vaut NULL.

Application et correction courte

1. Élèves de 3SI2 dont la moyenne atteint 10 : id_classe = 2 AND moyenne >= 10.

2. Élèves de Tunis ou de Sousse : ville = 'Tunis' OR ville = 'Sousse'.

3. Ville inconnue : ville IS NULL.

Tester sur le jeu initial et vérifier les noms attendus sans calcul d’agrégat.

BETWEEN, IN, LIKE et ORDER BY

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Filtres sur une seule table

```
SELECT nom, moyenne FROM eleve WHERE moyenne BETWEEN 10 AND 15;
SELECT nom, ville FROM eleve WHERE ville IN ('Tunis', 'Sfax');
SELECT nom, ville FROM eleve WHERE ville NOT IN ('Tunis', 'Sfax');
SELECT nom FROM eleve WHERE nom LIKE 'Tr%';
SELECT prenom FROM eleve WHERE prenom LIKE '_a%';
```

BETWEEN inclut les bornes. IN teste une liste de valeurs littérales ; ce chapitre n’utilise pas de requête imbriquée. LIKE utilise % pour zéro ou plusieurs caractères et _ pour un seul caractère. La sensibilité à la casse dépend de la collation.

Périodes et dates

```
SELECT nom, prenom, date_naissance FROM eleve
WHERE date_naissance BETWEEN '2009-01-01' AND '2009-12-31';
```

Ici la colonne est de type DATE : la borne supérieure couvre toute cette journée. Pour une colonne DATETIME, préférer un intervalle semi-ouvert qui inclut toutes les heures du dernier jour.

Trier le résultat

```
SELECT nom, prenom FROM eleve ORDER BY nom ASC, prenom ASC;
SELECT nom, moyenne FROM eleve ORDER BY moyenne DESC, nom ASC;
SELECT nom, prenom, moyenne FROM eleve
WHERE id_classe = 1 AND moyenne >= 10
ORDER BY moyenne DESC;
```

ASC est l’ordre croissant, DESC l’ordre décroissant. Le tri s’écrit après WHERE. Il change l’ordre du résultat, pas l’ordre physique garanti du stockage.

Ordre des clauses étudiées

```
SELECT DISTINCT colonnes
FROM une_table
WHERE condition
ORDER BY colonne ASC;
```

La notation colonnes est un repère, pas un nom réel à recopier. DISTINCT, WHERE et ORDER BY sont facultatifs selon la question.

Application

Afficher les élèves dont le nom commence par B, puis ceux de Tunis et Sfax par moyenne décroissante. Retrouver la liste des villes connues sans doublon, triée par ordre alphabétique.

TP 1 : mini-site du Lycée Rafèha

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Objectif

Assembler HTML5 et CSS3 dans un site de trois pages : accueil.html, inscription.html et ressources.html. Aucun traitement côté serveur n’est demandé.

Page d’accueil

Un header présente le lycée et une image de logo ; nav contient les liens vers les trois pages ; main regroupe une section et un article ; footer indique l’auteur. Ajouter une liste de langages et un tableau d’horaires avec caption et cellules d’en-tête.

Formulaire

Saisir nom, prénom, classe, adresse électronique et message. Utiliser label, required, email, select et textarea. Ajouter des choix radio et des cases à cocher. Pour cette activité, le bouton vérifie les données avec JavaScript sans enregistrer en base.

CSS demandé

Une feuille externe commune ; palette sobre ; classes réutilisées ; marges et padding cohérents ; border-box ; liens visibles au clavier ; ombre légère sur les cartes. Créer d’abord la mise en page avec float et clear. Ajouter une transition sur les boutons et un effet de focus sur les champs.

Corrigé de structure : extrait

```
<header><h1>Lycée Rafèha</h1></header>
<nav><a href="accueil.html">Accueil</a>
     <a href="inscription.html">Inscription</a>
     <a href="ressources.html">Ressources</a></nav>
<main><section><h2>Nos activités</h2>
  <article><h3>Projet Web</h3><p>Présentation du projet.</p></article>
</section></main>
<footer>Travail réalisé en 3ème SI</footer>
```

```
* { box-sizing: border-box; }
nav { float: left; width: 170px; }
nav a { display: block; padding: 0.5rem; }
main { margin-left: 190px; padding: 1rem; }
footer { clear: both; text-align: center; }
```

À rendre et à tester

Les trois pages et la feuille CSS. Tester la navigation, le clavier, les liens d’images, le zoom et une largeur de 390 pixels. Indiquer les améliorations nécessaires si la navigation fixe devient trop large sur téléphone.

TP 2 : formulaire contrôlé avec JavaScript

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Objectif

Afficher une confirmation locale après vérification d’un nom et d’une note. Les données ne sont ni envoyées à un serveur ni enregistrées en base.

```
<form id="fiche">
  <label for="nom">Nom</label>
  <input id="nom" required>
  <label for="note">Note</label>
  <input id="note" type="number" min="0" max="20"
         step="0.5" required>
  <button type="submit">Vérifier</button>
</form>
<p id="message" role="status"></p>

let formulaire = document.getElementById("fiche");
formulaire.addEventListener("submit", function (event) {
  event.preventDefault();
  let nom = document.getElementById("nom").value.trim();
  let champ = document.getElementById("note");
  let note = Number(champ.value);
  let sortie = document.getElementById("message");
  if (nom.length < 2 || champ.value === ""
      || !Number.isFinite(note) || note < 0 || note > 20) {
    sortie.textContent = "Vérifier le nom et la note.";
    return;
  }
  sortie.textContent = nom + " : "
    + (note >= 10 ? "Admis" : "Non admis");
});
```

Explications

preventDefault bloque l’envoi du formulaire. trim retire les espaces extérieurs ; Number convertit la valeur ; Number.isFinite vérifie la conversion. textContent affiche une saisie comme du texte, sans l’interpréter comme du HTML.

Extensions et tests

Ajouter un tableau de notes en mémoire, calculer une moyenne en JavaScript et afficher une horloge. Tester 0, 10, 20, une chaîne vide, une valeur hors limites et un nom contenant < ou &. Les contrôles HTML du navigateur peuvent empêcher l’événement submit pour un champ invalide : vérifier aussi directement la fonction de contrôle.

TP 3 : sélection simple — énoncé et corrigé

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Base de travail

Utiliser ecole_3si avec le jeu initial de cinq élèves. Chaque requête ci-dessous interroge une seule table.

Questions

1. Afficher tous les élèves.
2. Afficher uniquement les noms et prénoms.
3. Afficher les élèves dont la moyenne atteint 15.
4. Afficher les élèves de la classe 1, par moyenne décroissante.
5. Afficher les élèves de Tunis ou Sfax.
6. Afficher les élèves dont le nom commence par T.
7. Afficher les villes connues sans doublon et par ordre alphabétique.
8. Afficher les élèves nés en 2009.

Corrigé

```
USE ecole_3si;
SELECT * FROM eleve;
SELECT nom, prenom FROM eleve;
SELECT nom, prenom, moyenne FROM eleve WHERE moyenne >= 15;
SELECT nom, prenom, moyenne FROM eleve
WHERE id_classe = 1 ORDER BY moyenne DESC;
SELECT nom, prenom, ville FROM eleve
WHERE ville IN ('Tunis','Sfax');
SELECT nom, prenom FROM eleve WHERE nom LIKE 'T%';
SELECT DISTINCT ville FROM eleve
WHERE ville IS NOT NULL ORDER BY ville ASC;
SELECT nom, prenom, date_naissance FROM eleve
WHERE date_naissance BETWEEN '2009-01-01' AND '2009-12-31';
```

Résultats de contrôle

La question 3 conserve Amine et Sara. La question 4 place Sara, Amine puis Nour. La question 5 conserve Amine, Sara et Yassine. La question 6 conserve Trabelsi Sara. La question 7 donne Sfax, Sousse, Tunis. La question 8 conserve quatre élèves.

Limite de niveau

Ce TP n’utilise ni calcul d’agrégat, ni regroupement, ni jointure, ni sous-requête. Pour afficher le nom de la classe sans liaison de tables, consulter classe séparément : ne pas inventer une colonne nom_classe dans eleve.

Aide-mémoire HTML5, CSS3 et JavaScript

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

HTML5

Document : doctype, html, head, meta charset, title, body. Structure : header, nav, main, section, article, aside, footer. Texte : h1–h6, p, strong, em, br. Listes : ul, ol, li. Tableau : table, caption, tr, th, td. Formulaire : form, label, input, select, option, textarea, button. Médias : img, audio, video, source, figure, figcaption, map, area, iframe.

CSS3

Sélection : balise, .classe, #id, [attribut], :hover, :focus-visible. Boîte : width, height, padding, border, margin, box-sizing. Texte : font-family, font-size, font-weight, text-align, line-height. Présentation : color, background, border-radius, box-shadow. Mise en page : display, float, clear, position. Effets : filter, transform, transition, animation et @keyframes.

JavaScript

Déclarer avec let ; convertir avec Number ; vérifier avec Number.isFinite. Choisir avec if/else ou switch ; répéter avec for, while ou do...while. Définir une fonction avec function et renvoyer avec return. Sélectionner avec getElementById ou querySelector. Afficher du texte avec textContent. Écouter un événement avec addEventListener.

Modèles de raisonnement

1. Décrire les données et vérifier les entrées.
2. Initialiser les variables avant les calculs.
3. Définir les conditions et les limites.
4. Afficher un résultat lisible.
5. Tester un cas normal, un cas limite et une erreur.

Erreurs fréquentes

Confondre = et === ; additionner des chaînes au lieu de nombres ; oublier une variable hors de sa portée ; supposer qu’un indice de tableau commence à 1 ; écrire 150 px au lieu de 150px ; dupliquer un id ; oublier name pour une donnée à transmettre ; afficher une saisie avec innerHTML.

Travail personnel

Créer une carte mentale pour chaque langage. Les aides-mémoire illustrés du support initial sont remplacés ici par cette synthèse textuelle et les exemples des fiches, afin de conserver une bonne lisibilité à l’impression.

SQL : bilan de niveau et auto-évaluation

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Compétences attendues

Identifier tables, colonnes, lignes, clés primaires et étrangères. Choisir des types cohérents. Comprendre les contraintes. Créer ou modifier une table, insérer, modifier et supprimer des lignes en sécurité. Lire une seule table avec SELECT, DISTINCT, WHERE et ORDER BY.

Questions de révision

1. Pourquoi une CIN ou un téléphone doit-il généralement être textuel ?
2. Quelle différence entre DROP TABLE et DELETE FROM ?
3. Que risque un UPDATE sans WHERE ?
4. Pourquoi ville = NULL ne convient-il pas ?
5. BETWEEN inclut-il les bornes ?
6. Quelle différence entre LIKE 'A%' et LIKE 'A_' ?
7. DISTINCT modifie-t-il les données stockées ?
8. ORDER BY garantit-il un ordre de stockage physique ?

Réponses

1. Ce sont des identifiants, pas des quantités ; les zéros initiaux doivent être conservés.
2. DROP détruit la structure ; DELETE conserve la table et enlève des lignes.
3. Il modifie toutes les lignes satisfaisant les autres contraintes, donc potentiellement toute la table.
4. NULL exprime une information absente ; utiliser IS NULL.
5. Oui, ses deux bornes sont incluses.
6. A% accepte zéro ou plusieurs caractères après A ; A_ exige exactement un caractère après A.
7. Non, il élimine seulement des doublons dans le résultat.
8. Non, il trie le résultat de la requête.

Arrêt du cours

Les requêtes sur plusieurs tables, les sous-requêtes, les fonctions d’agrégation et les clauses de regroupement ne sont pas enseignées dans ce manuel. Les liens de l’application donnent accès à une plateforme commune à plusieurs niveaux : les sections au-delà du SELECT simple relèvent d’une poursuite d’étude.

Application finale

Sur une base personnelle de livres ou de produits, écrire cinq sélections à une seule table : projection, filtre numérique, liste de valeurs, motif de texte et tri. Expliquer chaque clause à l’oral.

Sources et utilisation du livre connecté

Ouvrir cette partie sur STI V2.0 BY A'e → | QR de lecture en bas de page

Supports de cette édition

PDF fourni : « 3stiCours STI HTML5 CSS3 JavaScript base de données.pdf », 74 pages. Comparaison avec les cours HTML5, CSS3, JavaScript et SQL de l’application STI V2.0 BY A'e. Les accès à l’application sont représentés par des QR codes et des libellés cliquables, sans adresse imprimée.

Adaptation pédagogique

Les exemples sont reformulés et corrigés pour l’impression. Le contenu de base du support initial est repris ; les parties SQL avancées sont retirées conformément au périmètre choisi. Les images mappées, le positionnement, les transitions et les propriétés d’animation sont conservés. Le livre ne reproduit pas toutes les animations et vidéos de la plateforme.

Repères techniques

Exemples actualisés pour 2026 ; let pour les variables JavaScript. SQL destiné à MySQL 8.0.16 ou ultérieur avec moteur InnoDB pour l’application effective des CHECK. Les schémas gestion_location et ecole_3si sont indépendants : sélectionner la bonne base avant chaque activité.

Un QR propre à chaque page

Le QR de chaque fiche identifie le livre de 3ème SI et son numéro de page. L’application retrouve ensuite la section concernée. Le QR de couverture ouvre le choix de ce manuel. La lecture directe du QR et l’import d’une photo nécessitent un code assez net ; la reconnaissance du texte sans QR n’est pas incluse.

Mode de lecture

Sur écran, utiliser le sommaire et le nom d’application cliquables. Sur papier, scanner le QR contrasté. Un filigrane QR très pâle marque l’édition ; il n’est pas le code de lecture garanti. Les adresses peuvent être révélées au scan ou par les propriétés des liens : elles ne sont pas secrètes.

Deux livres distincts

Choisir « 3ème SI — 2026 » dans le scanner ou laisser le QR sélectionner ce livre. Un même numéro de page ne renvoie pas au même contenu dans le manuel de 4ème SI. Les contrôles d’accès aux ressources en ligne restent ceux de la plateforme.